

■ TECHNICAL DOCUMENTATION

Customer Segmentation Dashboard

K-Means Clustering · PCA Visualisation · Streamlit · Python 3.9+

A complete professional reference — concepts, architecture, workflow & deployment.

Table of Contents

01	Definitions & Core Concepts
02	Code Examples & Implementation
03	Project Overview & Architecture
04	Step-by-Step Workflow
05	Technology Stack
06	Deployment Options
07	Future Roadmap & License

01 — Definitions & Core Concepts

Customer Segmentation

The process of dividing a customer base into distinct, homogeneous groups based on shared characteristics such as demographics, purchasing behaviour, or preferences. Enables businesses to tailor marketing messages, personalise offers, and allocate resources efficiently to maximise ROI.

Unsupervised Learning

A class of machine-learning tasks where the algorithm discovers hidden patterns or groupings in data without pre-labelled outcomes. Clustering is the most common unsupervised technique, used when the number or nature of categories is unknown in advance.

Clustering

Groups data points so that items within a cluster are more similar to each other than to those in other clusters. Similarity is typically measured with Euclidean distance. Common algorithms: K-Means, DBSCAN, Agglomerative, BIRCH, and Gaussian Mixture Models.

K-Means Clustering

A centroid-based iterative algorithm partitioning N data points into K clusters by minimising the total within-cluster sum of squared distances (inertia). Algorithm: (1) Randomly initialise K centroids, (2) Assign each point to the nearest centroid via Euclidean distance, (3) Recompute each centroid as the mean of its assigned points, (4) Repeat steps 2–3 until centroids no longer move significantly or max iterations reached. Key parameter: n_init — number of random initialisations to avoid local minima.

Inertia (Within-Cluster Sum of Squares)

The objective function K-Means minimises — sum of squared distances from each point to its assigned centroid. Lower inertia = tighter clusters. Use the Elbow Method: plot inertia vs K and pick the "elbow" where adding more clusters yields diminishing returns.

Principal Component Analysis (PCA)

A linear dimensionality-reduction technique that projects data onto the directions (principal components) of maximum variance. Components are orthogonal eigenvectors of the covariance matrix, ordered by explained variance. In this project, PCA compresses the selected features to 2 components for 2-D scatter-plot visualisation of cluster separation.

StandardScaler

Standardises features by removing the mean and scaling to unit variance: $z = (x - \mu) / \sigma$. Critical before K-Means because the algorithm uses Euclidean distance — unscaled features with large ranges would dominate the distance calculation and distort clusters.

Streamlit

Open-source Python framework that converts annotated scripts into shareable interactive web apps. Re-runs the script top-to-bottom on every user interaction. Uses `session_state` to persist data (e.g. clustering results) across re-runs without recomputing.

Silhouette Score

A metric for evaluating cluster quality ranging from -1 to $+1$. A score near $+1$ means points are well-matched to their own cluster and poorly matched to neighbours. Used to validate the chosen K without ground-truth labels.

02 — Code Examples & Implementation

Preprocessing — StandardScaler

Selects the chosen feature columns, copies the DataFrame slice to avoid mutation, fits StandardScaler on the data, and returns both the scaled array and the fitted scaler (needed later to inverse-transform for interpretability).

```
def preprocess_data(df, features): data = df[features].copy() # avoid mutating
original_scaler = StandardScaler() scaled = scaler.fit_transform(data) # zero mean,
unit variance return scaled, scaler
```

K-Means Clustering

Wraps scikit-learn KMeans. n_init=10 runs the algorithm 10 times with different seeds and keeps the best result (lowest inertia), reducing the chance of a poor local minimum.

```
def perform_kmeans(data, n_clusters=3, random_state=42): kmeans = KMeans(
n_clusters=n_clusters, random_state=random_state, n_init=10 # 10 random
initialisations ) labels = kmeans.fit_predict(data) return labels, kmeans # labels:
array of cluster IDs per row
```

PCA — 2-D Visualisation

Reduces the scaled feature matrix to 2 principal components. The explained variance ratio tells us how much information is retained — ideally above 70% combined.

```
def perform_pca(data, n_components=2): pca = PCA(n_components=n_components) pca_data =
pca.fit_transform(data) # pca.explained_variance_ratio_ shows variance captured per
component return pca_data, pca
```

Cluster Insights Extraction

After clustering, aggregate per-cluster statistics for the Insights page.

```
def get_cluster_insights(df, labels, features): df = df.copy() df['Cluster'] = labels
insights = {} for cluster_id in sorted(df['Cluster'].unique()): sub = df[df['Cluster']
== cluster_id] insights[cluster_id] = { 'count': len(sub), 'avg_income': sub['Annual
Income (k$)'].mean(), 'avg_spending': sub['Spending Score (1-100)'].mean(), } return
insights
```

Sample Data Generation

Two helper scripts ship with the repository for quick testing without real data:

- **generate_sample_data.py** — creates one synthetic 500-row dataset with randomised Age, Income, and Spending Score
- **generate_multiple_sample_data.py** — generates three CSVs: small (100 rows), mixed (300 rows), large (1000 rows)

```
python generate_multiple_sample_data.py
```

CSV files land in the project root and are loadable via the **Load Sample Data** button in the sidebar.

03 — Project Overview & Architecture

Overview

The Customer Segmentation Dashboard is a full-stack data application built on Streamlit. It accepts any CSV of customer records, preprocesses and clusters the data using K-Means, reduces dimensions with PCA for visualisation, and surfaces per-segment marketing insights. The goal is to convert raw transactional data into business-ready strategy with zero coding required from the end user.

Architecture Flow

CSV Upload → Preprocessing → K-Means → PCA → Dashboard → Insights → Export

Repository Structure

```
Customer-Segmentation-Dashboard/ ■■■ app.py # Main Streamlit app (pages + logic) ■■■  
run_app.py # Windows working-directory helper ■■■ requirements.txt # Pinned Python  
dependencies ■■■ Dockerfile # Container build (for Render / self-host) ■■■ Procfile #  
Heroku process definition ■■■ generate_sample_data.py # Single 500-row dataset  
generator ■■■ generate_multiple_sample_data.py # Multi-size dataset generator ■■■  
*.csv # Auto-generated sample datasets
```

Required CSV Columns

The app expects exactly these five columns (names are case-sensitive):

Column	Type	Description
CustomerID	int / str	Unique identifier per customer
Gender	str	Male / Female
Age	int	Customer age in years
Annual Income (k\$)	float	Annual income in thousands USD
Spending Score (1-100)	int	Mall-assigned score 1–100

Installation

```
# 1. Clone git clone  
https://github.com/ayushtripathi-45/Customer-Segmentation-Dashboard.git cd  
Customer-Segmentation-Dashboard # 2. Virtual environment (recommended) python -m venv  
venv source venv/bin/activate # Windows: venv\Scripts\activate # 3. Install  
dependencies pip install -r requirements.txt
```

04 — Step-by-Step Workflow

Step 01 — Upload Data

Select a CSV via the file uploader or click **Load Sample Data** in the sidebar. The app reads the file with pandas, validates the required columns, and previews the first 10 rows. Basic descriptive statistics (mean, std, min, max) are shown immediately.

Step 02 — Configure & Run Segmentation

Use the slider to choose the number of clusters K (2–6). Click **Run Clustering**. Internally: features are selected → StandardScaler normalises → KMeans fits (n_init=10) → cluster labels appended to DataFrame → PCA reduces to 2-D → all results stored in st.session_state.

Step 03 — Dashboard View

The Dashboard page renders: KPI summary cards (total customers, number of segments, avg income, avg spending score), a full data table with cluster column, and a bar chart showing customer count per cluster to visualise segment balance.

Step 04 — Insights Page

For every cluster the app shows: a descriptive label (e.g. "High Income – Low Spenders"), average Annual Income and Spending Score, customer count and percentage share, and 3–5 tailored marketing action suggestions (e.g. loyalty programmes, premium upsell).

Step 05 — Export Results

Download the clustered DataFrame as a **CSV** for CRM import. Download the PCA scatter plot as a **PNG** for reports and presentations. Both buttons appear at the bottom of the Segmentation page.

Running Locally

```
streamlit run app.py # opens http://localhost:8501 python run_app.py # Windows: sets  
working directory first
```

05 — Technology Stack

Library / Tool	Version	Role in Project
Python	3.9+	Core runtime — all logic, data processing, and web server
Streamlit	≥ 1.28	UI framework — pages, widgets, session state, re-run model
pandas	≥ 2.0	CSV I/O, DataFrame manipulation, groupby aggregations
numpy	≥ 1.24	Array ops, random seeds, numerical foundations for sklearn
scikit-learn	≥ 1.3	KMeans, PCA, StandardScaler — the core ML pipeline
matplotlib	≥ 3.7	Static scatter plots and bar charts rendered inside Streamlit
Docker	—	Container build for Render / self-hosted deployments

Why These Choices?

- **Streamlit** eliminates front-end complexity — pure Python produces a fully interactive web app.
- **scikit-learn** offers production-grade, well-tested implementations with a consistent API.
- **StandardScaler + KMeans** is the industry-standard pipeline for tabular customer segmentation.
- **PCA** provides an interpretable 2-D view that non-technical stakeholders can immediately grasp.
- **matplotlib** integrates natively with Streamlit via `st.pyplot()` — no extra server needed.

06 — Deployment Options

Streamlit Community Cloud

Connect your GitHub repo at share.streamlit.io. Set the main file to **app.py**. Free tier provides a permanent public URL (e.g. your-app.streamlit.app) with automatic redeploy on every git push. Best for quick sharing and demos.

Render.com

Add the supplied Dockerfile. Create a Web Service on Render and point to your repo. Render builds the image and exposes the app on a sub-domain. Custom domains and auto-deploy on push are supported. Free tier available with spin-down on inactivity.

```
docker build -t seg-dashboard . docker push ... # or connect GitHub directly
```

Docker / Self-hosted

Full control over environment. Suitable for on-premises or private cloud deployments. Map port 8501 to any external port. Add a reverse proxy (nginx / Caddy) for HTTPS.

```
docker build -t seg-dashboard . docker run -p 8501:8501 seg-dashboard
```

Heroku

Add a Procfile with the command below. Push to Heroku via git. The \$PORT env var is injected automatically by Heroku at runtime.

```
web: streamlit run app.py --server.port $PORT --server.headless true
```


07 — Future Roadmap & License

Planned Enhancements

- **Additional Algorithms** — DBSCAN (density-based, handles noise), Agglomerative Hierarchical (dendrogram view), Gaussian Mixture Models (soft assignments).
- **Interactive Visualisations** — Replace matplotlib with Plotly for zoomable, hoverable PCA and bar charts with cluster tooltips.
- **Elbow Method UI** — Auto-plot inertia vs K and silhouette scores to help users pick the optimal number of clusters.
- **Churn Prediction** — Add a supervised Random Forest model to flag at-risk customers within each segment.
- **Authentication & Multi-user** — Streamlit-Authenticator integration for team use with per-user dashboards.
- **Database Integration** — Connect to PostgreSQL or BigQuery for live customer data instead of CSV uploads.
- **Scheduled Reports** — Auto-generate and email PDF segment reports on a weekly/monthly cadence.

License & Contributions

Released under the **MIT License** — free to use, modify, and distribute with attribution.

Contributions are welcome: fork the repository, create a descriptive feature branch, write or update tests, and open a pull request with a clear description of changes.

Repository: github.com/ayushtripathi-45/Customer-Segmentation-Dashboard